

Compte Rendu de Travaux Pratiques — Simulation de Particules en C++

TP2 • TP3 • TP4 • TP5 • TP6

Abdellah BENSALÉM et Mohamed TROMBATI

2025–2026

Contents

Introduction	3
1 TP2 : Structures de données et simulation de particules	4
1.1 Objectifs	4
1.2 Classe Particule	4
1.3 Algorithme de Störmer-Verlet	4
1.4 Simulation du système solaire	4
1.5 Comparaison <code>vector</code> vs <code>list</code>	5
2 TP3 : Opérateurs et structures de données avancées	6
2.1 Objectifs	6
2.2 Classe Vecteur	6
2.2.1 Attributs et encapsulation	6
2.2.2 Opérateurs implémentés	6
2.3 Classe Particule	6
2.4 Classe Univers	6
2.4.1 Description	6
2.4.2 Calcul des forces gravitationnelles	6
2.5 Tests unitaires	7
2.6 Benchmarks de performance	7
3 TP4 : Découpage spatial et potentiel de Lennard-Jones	8
3.1 Objectifs	8
3.2 Potentiel de Lennard-Jones	8
3.2.1 Formule	8
3.2.2 Force dérivée	8
3.2.3 Rayon de coupure	8
3.3 Grille de cellules	8
3.3.1 Principe	8
3.3.2 Classe Cellule	9
3.3.3 Mise à jour des cellules	9
3.4 Optimisations implémentées	9
3.5 Tests unitaires	9
3.5.1 Infrastructure de test	9
3.5.2 Test de la valeur analytique des forces	9
3.5.3 Test de conservation de la quantité de mouvement	10
3.6 Export VTK pour ParaView	10

3.7	Simulation de collision	10
3.7.1	Paramètres	10
3.7.2	Description	10
4	TP5 & 6 : Conditions aux limites, Tests & Diagrammes UML	11
4.1	Conditions aux limites	11
4.1.1	Objectifs	11
4.1.2	Réflexion élastique (<code>type_border = 0</code>)	11
4.1.3	Conditions périodiques (<code>type_border = 1</code>)	11
4.1.4	Absorption (<code>type_border = 2</code>)	12
4.1.5	Forces de parois répulsives	12
4.1.6	Rescaling des vitesses	12
4.1.7	La fonction <code>check_validite</code>	13
4.1.8	Optimisations apportées:	13
4.2	Tests unitaires Google Test	13
4.2.1	Infrastructure et organisation	14
4.2.2	Tests de la classe <code>Vecteur</code>	14
4.2.3	Tests de la classe <code>Particule</code>	14
4.2.4	Tests de la classe <code>Univers</code>	14
4.2.5	Tests des conditions aux limites (<code>WallsTest</code>)	15
4.2.6	Tests physiques et forces LJ	15
4.3	Diagrammes UML	15
4.3.1	Diagramme de classes	15
4.3.2	Diagramme de cas d'utilisation	15
4.3.3	Diagramme de séquence	16
4.3.4	Diagramme de transitions d'état	16
4.4	Simulation de collision avec gravité (TP6)	16
4.4.1	Paramètres	16
4.4.2	Description et observations	16
	Conclusion	18

Introduction

Ce compte rendu présente les travaux pratiques réalisés dans le cadre du cours *C++ pour les mathématiques appliquées*. L'objectif global est de développer, par étapes successives, une simulation de particules capable de modéliser des interactions physiques.

Chaque TP enrichit le projet d'une couche supplémentaire :

- **TP2** introduit la classe `Particule` et l'algorithme de Störmer-Verlet pour la simulation du système solaire ;
- **TP3** structure le code en classes indépendantes (`Vecteur`, `Particule`, `Univers`) avec surcharge d'opérateurs ;
- **TP4** introduit un découpage spatial par grille de cellules, le potentiel de Lennard-Jones, et l'export VTK pour ParaView ;
- **TP5 et 6** ajoutent un ensemble complet de tests Google Test, et des diagrammes UML, et les conditions aux limites .

1 TP2 : Structures de données et simulation de particules

1.1 Objectifs

Le TP2 a pour objectif de créer une première simulation de système de particules en C++, d'explorer les conteneurs standards de la STL (`std::vector` et `std::list`), et d'implémenter l'algorithme de Störmer-Verlet pour l'intégration numérique des équations du mouvement.

1.2 Classe Particule

La classe `Particule` encapsule toutes les propriétés d'une particule physique. Les attributs sont privés et accessibles via des getters et setters :

- **Position** (`x`, `y`, `z`) : coordonnées spatiales en double précision
- **Vitesse** (`vitesse_x`, `vitesse_y`, `vitesse_z`) : composantes du vecteur vitesse
- **Force** (`force_x`, `force_y`, `force_z`) : force résultante appliquée à la particule
- **Masse** : scalaire double, avec validation (exception si négative ou nulle)
- **Identifiant** et **catégorie** : métadonnées de la particule

Un soin particulier est apporté à la validation : le setter `setMasse()` lève une exception `std::invalid_argument` si la masse est inférieure ou égale à zéro.

```
void setMasse(double m) {
    if (m <= 0)
        throw std::invalid_argument("La masse doit etre positive.");
    masse = m;
}
```

1.3 Algorithme de Störmer-Verlet

L'algorithme de Störmer-Verlet (vitesse-Verlet) est une méthode d'intégration numérique d'ordre 2. Il se déroule en trois étapes par pas de temps Δt :

$$\mathbf{x}(t + \Delta t) = \mathbf{x}(t) + \mathbf{v}(t) \Delta t + \frac{1}{2} \mathbf{a}(t) \Delta t^2$$

$$\mathbf{F}(t + \Delta t) = \text{calculerForces}(\mathbf{x}(t + \Delta t))$$

$$\mathbf{v}(t + \Delta t) = \mathbf{v}(t) + \frac{1}{2}(\mathbf{a}(t) + \mathbf{a}(t + \Delta t)) \Delta t$$

Cet algorithme conserve mieux l'énergie totale du système sur de longues simulations par rapport à l'intégrateur d'Euler simple, ce qui est crucial pour les orbites planétaires.

1.4 Simulation du système solaire

La simulation comporte quatre corps :

Corps	Masse	Vitesse initiale
Soleil	1.0	(0, 0, 0)
Terre	3×10^{-6}	(-1, 0, 0)
Jupiter	9.55×10^{-4}	(-0.425, 0, 0)
Halley	10^{-14}	(0, 0.0296, 0)

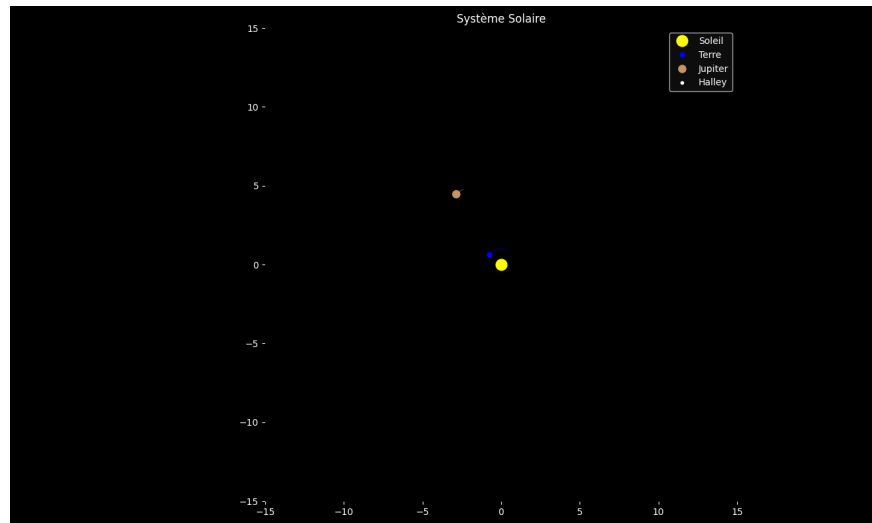


Figure 1: Simulation du système solaire

Les paramètres sont $\Delta t = 0.015$ et $t_{\text{end}} = 468.5$. Les positions sont sauvegardées dans `positions.txt`. La visualisation a été faite en Python avec pandas.

1.5 Comparaison `vector` vs `list`

Une comparaison de performances est réalisée pour l'insertion de N particules aléatoires :

Conteneur	Avantages	Inconvénients	Cas d'usage
<code>std::vector</code>	Accès aléatoire $O(1)$, cache-friendly	Insertion en milieu $O(N)$	Simulation (accès par indice)
<code>std::list</code>	Insertion $O(1)$ partout, pas de réallocation	Pas d'accès aléatoire, mauvaise localité mémoire	Suppressions fréquentes en milieu

Le `std::vector` s'avère systématiquement plus rapide pour notre usage car les particules sont accédées par indice à chaque pas de temps.

2 TP3 : Opérateurs et structures de données avancées

2.1 Objectifs

Le TP3 restructure et enrichit le code du TP2 en séparant le projet en plusieurs classes indépendantes avec leurs propres fichiers. Il introduit la classe `Vecteur` avec surcharge d'opérateurs et la classe `Univers` pour gérer un ensemble de particules.

2.2 Classe Vecteur

2.2.1 Attributs et encapsulation

Les attributs `x`, `y`, `z` sont privés. L'accès est géré par des getters `const` pour la lecture et des setters pour la modification.

2.2.2 Opérateurs implémentés

Opérateur	Description	Exemple
<code>operator+</code>	Addition vectorielle	<code>v1 + v2</code>
<code>operator-</code>	Soustraction vectorielle	<code>v1 - v2</code>
<code>operator*</code>	Multiplication par un scalaire	<code>v * 2.0</code>
<code>operator/</code>	Division par un scalaire	<code>v / m</code>
<code>operator+=</code>	Addition en place	<code>v += acc * dt</code>
<code>operator-=</code>	Soustraction en place	<code>force -= F</code>
<code>norm()</code>	Norme euclidienne	<code>rij.norm()</code>

2.3 Classe Particule

La classe `Particule` utilise désormais trois instances de `Vecteur` (`position`, `vitesse`, `force`) au lieu de neuf scalaires séparés. Deux versions de getters sont fournies pour chaque vecteur :

```
// Getter const pour la lecture
const Vecteur& getPosition() const { return position; }

// Getter non-const : retourne une reference pour += et -=
Vecteur& getPosition() { return position; }
```

2.4 Classe Univers

2.4.1 Description

La classe `Univers` contient une collection de particules et gère la simulation gravitationnelle. Elle intègre un vecteur de particules, la dimension de l'espace (1D, 2D ou 3D), le temps courant `t`, et les méthodes `calcul_forces()`, `avancer()` et `afficher()`.

2.4.2 Calcul des forces gravitationnelles

L'interaction gravitationnelle entre chaque paire (i, j) est calculée avec un paramètre de softening $\varepsilon = 0.05$ pour éviter les singularités à courte distance :

$$F_{ij} = \frac{G m_i m_j}{r_{ij}^3 + \varepsilon^2} \cdot \mathbf{r}_{ij}$$

La troisième loi de Newton est exploitée en ne calculant chaque paire (i, j) qu'une seule fois (boucle avec $j > i$), divisant le nombre d'opérations par 2.

2.5 Tests unitaires

Des tests Google Test sont écrits pour chaque méthode de la classe **Vecteur** (constructeurs, norme, les quatre opérations arithmétiques, les opérateurs cumulés), ainsi que pour la conservation de l'impulsion dans l'**Univers**. On peut lancer la commande `./tests` pour les afficher tous, ou les filtrer avec `--gtest_filter`.

2.6 Benchmarks de performance

La complexité du calcul des forces est $O(N^2)$. Les benchmarks montrent la croissance quadratique attendue :

k	$N = (2^k)^3$	Paires évaluées	Temps approximatif
3	512	130 816	\$ \$12 ms
4	4 096	8 386 560	\$ \$700 ms
5	32 768	536 854 528	\$ \$47 s
6	262 144	\$>\$34 milliards	Prohibitif

Le facteur d'accélération apporté par Newton 3 est d'environ $2\times$ par rapport à la version naïve. Pour aller plus loin, une structure spatiale hiérarchique (grille de cellules) devient indispensable, ce qui est l'objet du TP4.

3 TP4 : Découpage spatial et potentiel de Lennard-Jones

3.1 Objectifs

Le TP4 introduit deux avancées majeures : le potentiel de Lennard-Jones comme modèle d'interaction, et un découpage de l'espace en grille de cellules pour réduire la complexité de $O(N^2)$ à quasi-linéaire grâce à un rayon de coupure.

3.2 Potentiel de Lennard-Jones

3.2.1 Formule

Le potentiel de Lennard-Jones décrit l'interaction entre deux particules en fonction de leur distance r_{ij} :

$$U(r_{ij}) = 4\varepsilon \left[\left(\frac{\sigma}{r_{ij}} \right)^{12} - \left(\frac{\sigma}{r_{ij}} \right)^6 \right]$$

Le terme en r^{-12} modélise la répulsion à courte distance et le terme en r^{-6} modélise l'attraction à longue distance. Les paramètres utilisés sont $\varepsilon = 5.0$ et $\sigma = 1.0$.

3.2.2 Force dérivée

La force sur la particule i est dérivée du potentiel global :

$$\mathbf{F}_{ij} = \frac{24\varepsilon}{r_{ij}^2} \left[\left(\frac{\sigma}{r_{ij}} \right)^6 - 2 \left(\frac{\sigma}{r_{ij}} \right)^{12} \right] \mathbf{r}_{ij}$$

La distance d'équilibre est $r^* = 2^{1/6} \sigma \approx 1.122 \sigma$, point où la force est nulle. En dessous de r^* , la force est répulsive ; au-dessus, elle est attractive.

3.2.3 Rayon de coupure

Au-delà de $r_{\text{cut}} = 2.5 \sigma$, le potentiel est négligé :

$$U(r_{ij}) = \begin{cases} 4\varepsilon \left[\left(\frac{\sigma}{r_{ij}} \right)^{12} - \left(\frac{\sigma}{r_{ij}} \right)^6 \right] & r_{ij} \leq r_{\text{cut}} \\ 0 & r_{ij} > r_{\text{cut}} \end{cases}$$

3.3 Grille de cellules

3.3.1 Principe

L'espace de simulation est découpé en une grille tensorielle de cubes. Le nombre de cellules dans chaque direction est :

$$n_{cd} = \left\lfloor \frac{L_d}{r_{\text{cut}}} \right\rfloor$$

Chaque cellule connaît ses voisines (jusqu'à 27 en 3D). Pour calculer les forces d'une particule, on n'examine que les particules dans les cellules voisines, garantissant que toutes les particules à distance $\leq r_{\text{cut}}$ sont considérées.

3.3.2 Classe Cellule

La classe `Cellule` encapsule :

- La liste des indices des particules (`std::vector<int>`)
- Son centre géométrique (`Vecteur`)
- Un tableau précalculé des indices de ses cellules voisines (`std::array<int,27>`)
- Le nombre de voisins valides (`nb_voisins`)

Les attributs sont privés avec getters et setters. Les voisins sont précalculés à l'initialisation.

3.3.3 Mise à jour des cellules

À la fin de chaque pas de temps, `mettre_a_jour_cellules()` vide toutes les cellules puis réaffecte chaque particule à sa cellule courante selon sa position. Cette opération est $O(N)$ et est appelée deux fois par pas de temps.

3.4 Optimisations implémentées

Optimisation	Description	Gain estimé
Newton 3	Chaque paire (i, j) calculée une seule fois	$2\times$
Grille + r_{cut}	Seules les cellules voisines sont examinées	$O(N^2) \rightarrow O(N)$
Calcul sur r^2	Évite le <code>sqrt()</code> pour la comparaison avec r_{cut}^2	$\sim 1.5\times$
Sans <code>pow()</code>	<code>sr2*sr2*sr2</code> au lieu de <code>pow(sr2,3)</code>	$\sim 2\times$
Compilation -O2/-O3	Optimisations du compilateur	5 à $10\times$

3.5 Tests unitaires

3.5.1 Infrastructure de test

Les tests sont organisés en quatre fichiers indépendants :

- `test_vecteur.cxx` : constructeurs, norme, opérations arithmétiques
- `test_particule.cxx` : constructeur par défaut, setters/getters, modification par référence
- `test_univers.cxx` : dimensions de grille, affectation des cellules, voisins, conservation de la quantité de mouvement
- `test_force.cxx` : distance d'équilibre LJ, signe de la force, Newton 3, rayon de coupure, valeur analytique

3.5.2 Test de la valeur analytique des forces

Un test particulièrement rigoureux compare la force calculée par simulation avec la valeur analytique exacte pour deux particules alignées sur l'axe X à distance $r = 1.1\sigma$:

```
double Fx_sim = ps[0].getForce().getX();
double Fx_anal = force_lj_analytique(eps, sig, r);
check(approx(Fx_sim, Fx_anal, 1e-8),
      "Fx simulee == Fx analytique pour r=1.1*sigma");
```

3.5.3 Test de conservation de la quantité de mouvement

Le test vérifie que la somme des quantités de mouvement reste constante après 10 pas de temps :

$$\left\| \sum_i m_i \mathbf{v}_i(t + 10 \Delta t) - \sum_i m_i \mathbf{v}_i(t) \right\| < 10^{-8}$$

3.6 Export VTK pour ParaView

Chaque frame est sauvegardée dans un fichier `.vtp` (VTK PolyData XML) contenant :

- Les positions 3D de toutes les particules
- La norme de la vitesse (utilisée pour la colorisation dans ParaView)
- Le vecteur vitesse complet (3 composantes)
- L'identifiant de chaque particule

Les fichiers sont stockés dans le répertoire `vtk_output` dans `build`.

3.7 Simulation de collision

3.7.1 Paramètres

Paramètre	Valeur	Description
N_{total}	8 000	1 600 (rouge) + 6 400 (bleu)
ε	5.0	Profondeur du potentiel LJ
σ	1.0	Distance caractéristique LJ
r_{cut}	2.5σ	Rayon de coupure
Δt	5×10^{-5}	Pas de temps
t_{end}	19.5	Durée totale
$\mathbf{v}_0$ (rouge)	$(0, -10, 0)$	Vitesse initiale du bloc rouge
d_{inter}	$2^{1/6} \sigma$	Distance d'équilibre LJ

3.7.2 Description

Un bloc rouge de 40×40 particules tombe avec une vitesse initiale de $\mathbf{v} = (0, -10, 0)$ sur un bloc bleu stationnaire de 160×40 particules. La collision génère des vitesses élevées dans la zone d'impact, visualisées par une colorisation rouge dans ParaView.

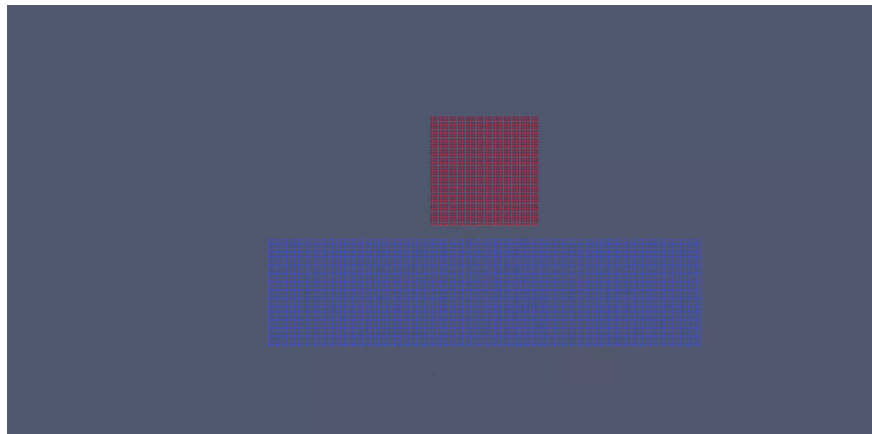


Figure 2: Début de simulation (TP4)

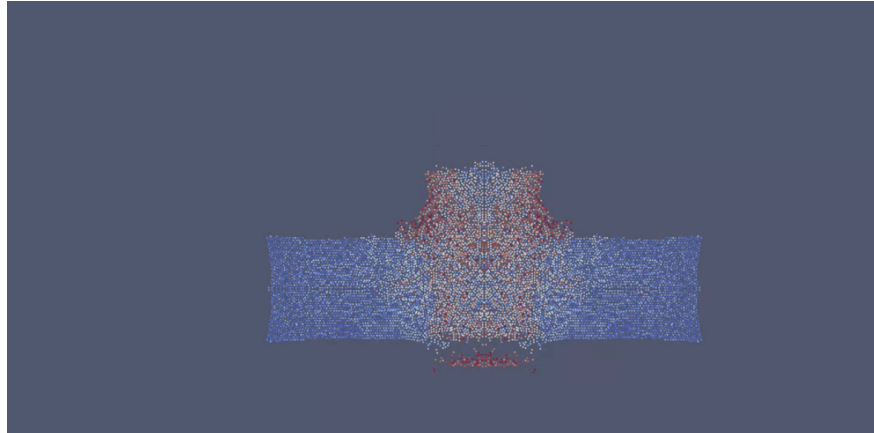


Figure 3: Après quelques instants (TP4)

4 TP5 & 6 : Conditions aux limites, Tests & Diagrammes UML

4.1 Conditions aux limites

4.1.1 Objectifs

Le TP6 enrichit la classe `Univers` avec la gestion des frontières du domaine de simulation. Trois comportements sont disponibles via le paramètre `type_border` :

- `type_border = 0` : **réflexion élastique** — les particules rebondissent sur les parois
- `type_border = 1` : **conditions périodiques** — une particule sortant d'un côté réapparaît du côté opposé
- `type_border = 2` : **absorption** — les particules sortant du domaine sont supprimées

4.1.2 Réflexion élastique (`type_border = 0`)

La position est recalée à l'intérieur du domaine et la composante de vitesse perpendiculaire à la paroi est inversée, conservant l'énergie cinétique totale. en mettant aussi une distance de sécurité de 10^{-6}

```
Vecteur& pos = p.getPosition();
Vecteur& vit = p.getVitesse();
if (pos.getX() < 0.0) {
    if (type_border == 0) {
        pos.setX(eps_wall);
        vit.setX(std::abs(vit.getX()));
    }
    //...
}
else if (pos.getX() > L[0]) {
    if (type_border == 0) {
        pos.setX(L[0] - eps_wall);
        vit.setX(-std::abs(vit.getX()));
    }
    //...
}
```

4.1.3 Conditions périodiques (`type_border = 1`)

Les particules réapparaissent du côté opposé avec la même vitesse, simulant un milieu infini et homogène.

```

if (pos.getX() < 0.0) {
    //...
    else if (type_border == 1) {
        pos.setX(pos.getX() + L[0]);
    }
    //...
}
else if (pos.getX() > L[0]) {
    //...
    else if (type_border == 1) {
        pos.setX(pos.getX() - L[0]);
    }
    //...
}

```

4.1.4 Absorption (type_border = 2)

Les particules hors domaine sont retirées du vecteur de particules.

```

if (pos.getX() < 0.0) {
    //...
    else if (type_border == 2) {
        p.setMasse(0);
    }
}
//...
// Suppression si absorption
if (type_border == 2) {
    particules.erase(
        std::remove_if(particules.begin(), particules.end(),
            [](const Particule& p) { return p.getMasse() == 0; }),
        particules.end()
    );
}

```

4.1.5 Forces de parois répulsives

La méthode `ajouter_forces_parois()` applique une force répulsive de type Lennard-Jones sur les particules proches des parois. Une distance minimale de 0.1σ est imposée pour éviter les singularités numériques.

4.1.6 Rescaling des vitesses

La méthode `rescaler_vitesses(Ec_cible)` ajuste les vitesses pour atteindre une énergie cinétique cible :

$$\beta = \sqrt{\frac{E_{c,cible}}{E_c}}$$

```

const double Ec = energie_cinetique();
if (Ec < 1e-12) {
    std::cerr << " Energie cinetique quasi-nulle, rescaling ignoré\n";
    return;
}
const double beta = std::sqrt(Ec_cible / Ec);

```

```

for (auto& p : particules) {
    Vecteur& v = p.getVitesse();
    v.setX(v.getX() * beta);
    v.setY(v.getY() * beta);
    v.setZ(v.getZ() * beta);
}

```

4.1.7 La fonction `check_validite`

Le mécanisme de gestion des erreurs mis en place présente plusieurs avantages. Tout d'abord, il permet de détecter rapidement des anomalies numériques telles que des valeurs non définies (NaN) ou infinies dans les positions, les vitesses ou l'énergie cinétique. Cela évite la propagation silencieuse d'erreurs dans la simulation et facilite grandement le débogage. De plus, l'utilisation d'exceptions permet d'interrompre proprement l'exécution du programme en cas de problème grave, garantissant ainsi une meilleure robustesse du code.

```

void Univers::check_validite() {
    for (const auto& p : particules) {
        const Vecteur& pos = p.getPosition(); // référence, pas de copie
        if (!std::isfinite(pos.getX()) ||
            !std::isfinite(pos.getY()) ||
            !std::isfinite(pos.getZ()))
            throw std::runtime_error("Position non finie détectée, explosion numérique probable");
    }
    if (!std::isfinite(energie_cinetique()))
        throw std::runtime_error("Énergie cinétique non finie, explosion numérique probable");
}

```

Cependant, ce mécanisme possède également certaines limites. D'une part, les vérifications ajoutent un coût de calcul supplémentaire, en particulier si elles sont effectuées à chaque itération. D'autre part, il s'agit d'un mécanisme purement détectif : il identifie les erreurs mais ne propose pas de stratégie pour les corriger automatiquement. Par ailleurs, le choix des seuils (par exemple pour éviter des distances trop faibles) reste arbitraire et peut influencer le comportement de la simulation.

Enfin, on peut compléter ce dispositif par un suivi de grandeurs physiques globales, comme l'énergie cinétique ou l'énergie totale, qui constitue un indicateur pertinent de la stabilité numérique du système. Cela permet de détecter des dérives sans nécessairement interrompre immédiatement la simulation.

4.1.8 Optimisations apportées:

Plusieurs optimisations ont été apportées au code afin de réduire les coûts inutiles en mémoire et en calcul. Dans `ajouter_forces_parois()`, les copies par valeur de `pos` et `force` ont été remplacées par des références directes, et les quatre blocs identiques de calcul du potentiel de paroi ont été factorisés en une unique `beta` locale. Dans `ajouter_gravite()`, l'appel redondant à `setForce()` sur une référence a été supprimé. Dans `avancer()`, le vecteur `forces_old` déclaré static — source potentielle de bugs en cas d'instances multiples — a été remplacé par une variable locale, et les objets `Vecteur` temporaires créés à chaque itération ont été éliminés au profit d'une arithmétique scalaire directe. La méthode `energie_cinetique()` calculait inutilement une racine carrée via `norm()` avant de l'élever au carré ; elle opère désormais directement sur le produit scalaire de vecteur avec lui même. Le constructeur d'`Univers` utilise maintenant `std::move` pour transférer le vecteur longueurs dans le membre `L` sans copie superflue. Enfin, les constructeurs de copie et destructeurs écrits manuellement dans `Vecteur` et `Particule` ont été remplacés par `= default`, laissant au compilateur le soin de générer des versions optimisées.

4.2 Tests unitaires Google Test

4.2.1 Infrastructure et organisation

les tests peuvent être lancés dans le terminal avec la commande `ctest -verbose` dans le répertoire build. On peut les filtrer avec `-gtest_filter=NomClasseTest.fonctionnalité` si on travaille dans le répertoire build.

Fichier	Suite	Nb tests	Couverture
test_vecteur.cxx	VecteurTest	11	Constructeurs, norme, opérateurs, cas limites
test_particule.cxx	ParticuleTest	7	Getters/setters, copie, flux, références
test_univers.cxx	UniversTest / WallsTest / PhysicsTest	9	Grille, voisinage, conditions limites, rescaling
test_force.cxx	ForcesLJTest	5	Équilibre LJ, signe, Newton 3, rcut, valeur analytique

4.2.2 Tests de la classe Vecteur

VecteurTest::Constructeurs — vérifie que le constructeur par défaut initialise $(0, 0, 0)$ et que le constructeur paramétrique stocke correctement les composantes.

VecteurTest::Norme — teste la norme euclidienne : $(3, 4, 0) \rightarrow 5$, vecteur nul $\rightarrow 0$, $(1, 1, 1) \rightarrow \sqrt{3}$.

VecteurTest::DivisionByZero — vérifie qu’une exception est levée lors d’une division par zéro.

VecteurTest::CopyAndAssignment et LargeValues — teste la stabilité numérique avec de très grandes valeurs (10^6) et l’indépendance des copies.

4.2.3 Tests de la classe Particule

ParticuleTest::ForceMutable et ForceReferenceConsistence — vérifient que le getter non-const retourne bien une référence modifiable :

```
p.getForce() += Vecteur(3, 0, 0);
EXPECT_NEAR(p.getForce().getX(), 3.0, 1e-10);
```

ParticuleTest::CycleDeVie — vérifie l’absence d’aliasing entre copie et original.

4.2.4 Tests de la classe Univers

UniversTest::GrilleDimensions — domaine $10 \times 10 \times 1$ avec $r_{\text{cut}} = 2.5$ doit produire 16 cellules.

UniversTest::Voisins — vérifie le nombre de voisins en 2D sur une grille 3×3 :

Position	Index	Voisins attendus
Coin $(0, 0, 0)$	0	4
Bord milieu $(1, 0, 0)$	1	6
Centre $(1, 1, 0)$	4	9

UniversTest::ConservationQuantiteMouvement — après 10 pas de temps :

$$\left\| \sum_i m_i \vec{v}_i(t + 10 \Delta t) - \sum_i m_i \vec{v}_i(t) \right\| < 10^{-8}$$

4.2.5 Tests des conditions aux limites (WallsTest)

WallsTest::ReflexionX — particule à $x = -1$ avec $v_x < 0$: après réflexion, $x > 0$ et $v_x > 0$.

WallsTest::PeriodiqueX — particule à $x = 11$ (hors $[0, 10]$) : après périodicité, $x < 10$.

WallsTest::Absorption — particule à $x = -1$: le vecteur de particules doit être vide après absorption.

4.2.6 Tests physiques et forces LJ

PhysicsTest::RescalingWorks — masse 1, vitesse 10 en X ($E_c = 50$) : après `rescaler_vitesses(1.0)`, $E_c = 1.0$ à 10^{-6} près.

ForcesLJTest::ValeurAnalytique — compare la force simulée à la valeur analytique pour $r = 1.1 \sigma$ à 10^{-8} près :

```
double Fx_sim = ps[0].getForce().getX();
double Fx_anal = force_lj_analytique(eps, sig, r);
EXPECT_NEAR(Fx_sim, Fx_anal, 1e-8);
```

4.3 Diagrammes UML

4.3.1 Diagramme de classes

Le simulateur est organisé en trois paquetages aux responsabilités bien séparées.

Le paquetage **Core Physics** contient **Vecteur** et **Particule**. **Vecteur** expose les attributs privés **x**, **y**, **z** ainsi que les opérations arithmétiques de base et le calcul de norme. **Particule** encapsule trois instances de **Vecteur** (position, vitesse, force), une masse **m** et un identifiant **id**.

Le paquetage **Spatial Optimization** contient **Cellule**, qui stocke la liste des identifiants des particules qu'elle contient (`std::vector<int>`), son centre géométrique (**Vecteur**), et un tableau précalculé de ses voisins (`std::array<int, 27>`).

Le paquetage **Orchestrator** contient **Univers**, qui coordonne l'ensemble de la simulation. En plus des méthodes présentes depuis TP4 (`avancer`, `calculer_forces_lj`, `mettre_a_jour_cellules`, `sauvegarderVTK`), le TP6 y a ajouté `appliquer_conditions_limites()`, `ajouter_forces_parois()`, `rescaler_vitesses(Ec_cible)` et `check_validite()`. L'attribut `type_border` contrôle le comportement aux frontières (0 = réflexion, 1 = périodique, 2 = absorption).

Les relations sont les suivantes : **Univers** **compose** un vecteur de **Particule** et un vecteur de **Cellule** ; chaque **Particule** **compose** trois instances de **Vecteur** ; chaque **Cellule** **réfère** des **Particule** via leurs identifiants entiers (association faible) et **compose** un **Vecteur** centre.

4.3.2 Diagramme de cas d'utilisation

Trois acteurs interagissent avec le système.

L'Utilisateur (Scientifique) lance la simulation. Cette action inclut obligatoirement (`<<include>>`) le chargement des paramètres (δt , r_{cut} , ε), le calcul des forces LJ et gravitationnelles, et l'application des conditions aux limites. La sauvegarde VTK et le rescaling des vitesses sont des extensions (`<<extend>>`) déclenchées conditionnellement — respectivement toutes les 50 itérations (après `step > 100`) et toutes les 1000 itérations (après `step > 5000`).

Le Développeur (Testeur) exécute les tests unitaires. Deux spécialisations héritent de ce cas d'utilisation : la vérification de la conservation de l'énergie et la vérification du voisinage de la grille.

ParaView est un acteur externe qui consomme les fichiers VTK produits par la sauvegarde.

4.3.3 Diagramme de séquence

Le diagramme décrit les interactions entre objets lors de l'exécution de `main()`.

L'utilisateur lance la simulation. `main()` appelle `avancer(dt, G=-12)` sur `Univers`, qui exécute la boucle de Verlet. À chaque pas de temps : les positions sont mises à jour (étape 1 de Verlet) ; les cellules sont vidées puis réaffectées via `mettre_a_jour_cellules()` ; les forces LJ et gravitationnelles sont calculées par paire de cellules voisines via `calculer_forces_lj()` ; les conditions aux limites sont appliquées via `appliquer_conditions_limites()` ; enfin les vitesses sont mises à jour (étape 2 de Verlet).

À chaque itération de la boucle principale de `main()`, en dehors de la boucle Verlet interne on appelle `check_validite()` toutes les 100 itérations. Le rescaling est déclenché à partir de l'itération 5000, puis toutes les 1000 itérations, en multipliant les vitesses par $\beta = \sqrt{E_c^D/E_c}$ — ce qui **ralentit** les particules pour maintenir $E_c \leq E_c^D$. L'export VTK est effectué toutes les 50 itérations après `step > 100`.

4.3.4 Diagramme de transitions d'état

Le cycle de vie de la simulation comporte trois états principaux.

L'état **Initialisation** comprend trois sous-états séquentiels : allocation mémoire, mise en place de la grille (calcul de `n_cubes`), puis placement initial des particules (mer et disque).

L'état **Simulation** est une boucle pilotée par la condition $t < T_{\text{END}}$. Chaque itération comprend le calcul des forces LJ et gravitationnelles, l'application des conditions aux limites, la mise à jour des positions (Verlet), la mise à jour de la grille spatiale, puis un test conditionnel de rescaling (actif uniquement si `step > 5000` et `step % 1000 = 0`, ramenant E_c vers E_c^D par ralentissement).

L'état **SauvegardeVTK** est déclenché périodiquement depuis la boucle (`step > 100` et `step % 50 = 0`). Après écriture et fermeture du fichier, la simulation reprend.

4.4 Simulation de collision avec gravité (TP6)

4.4.1 Paramètres

La simulation modélise un disque ($N_1 \approx 395$ particules, masse $m = 3$) tombant sur une mer stationnaire ($N_2 \approx 16\,772$ particules, masse $m = 1$) sous l'effet d'un champ gravitationnel uniforme. Le nombre total réel de particules générées est **17 167**, l'espacement étant fixé à 1.00776σ pour approcher les valeurs cibles.

Paramètre	Valeur	Description
$L_1 \times L_2$	250×180	Dimensions du domaine
ε, σ	1.0, 1.0	Paramètres LJ
Espacement	1.00776σ	Distance inter-particules initiale
m_{disque}	3.0	Masse des particules du disque
m_{mer}	1.0	Masse des particules de la mer
$\mathbf{v}_0$ (disque)	(0, -10, 0)	Vitesse initiale du disque
r_{cut}	2.5σ	Rayon de coupure
δt	0.0005	Pas de temps
G	-12	Champ gravitationnel (axe y)
E_c^D	$0.005 \times 17\,622$	Énergie cinétique cible
Rescaling	<code>step > 5000</code> , puis $\times 1000$	Déclenché tardivement
t_{end}	29.5	Durée totale

4.4.2 Description et observations

Le disque, plus dense ($m = 3$), tombe sous l'effet de la gravité ($G = -12$) et percute la mer stationnaire. Le rescaling est activé tardivement à partir de l'itération 5000, puis appliqué toutes les 1000 itérations via le facteur $\beta = \sqrt{E_c^D/E_c}$. Ce mécanisme **ralentit** les particules quand l'énergie cinétique dépasse la cible E_c^D ,

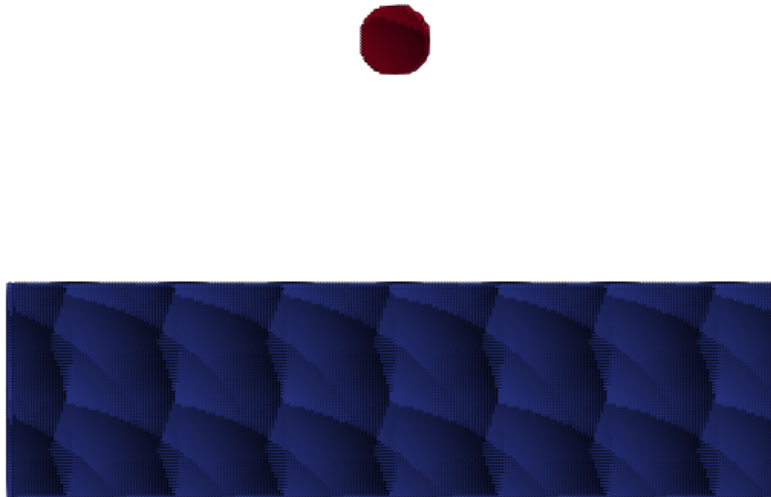


Figure 4: Début de simulation

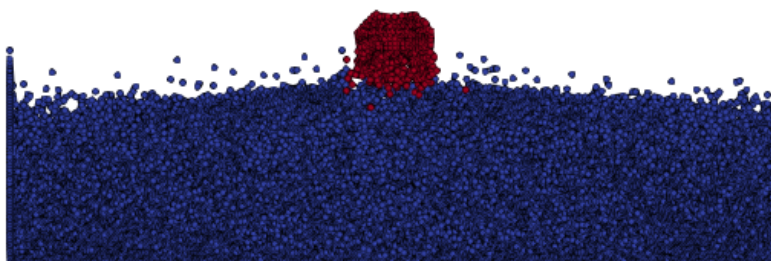


Figure 5: En cours de simulation — impact et déformation

limitant la divergence des vitesses en fin de simulation. Le déclenchement tardif permet à la collision de se développer librement dans un premier temps, ce qui produit un impact plus dynamique et une déformation visible de la mer, sans pour autant mener à une dispersion totale du système.

Conclusion

Ces travaux pratiques ont permis de construire progressivement une simulation de particules complète en C++ moderne.

Sur le plan **algorithmique**, le passage de l'intégrateur d'Euler à Störmer-Verlet garantit une conservation de l'énergie d'ordre 2, et l'introduction de la grille de cellules réduit la complexité du calcul des forces de $O(N^2)$ à un comportement quasi-linéaire grâce au rayon de coupure r_{cut} .

Sur le plan **génie logiciel**, le code illustre les bonnes pratiques C++ : encapsulation, surcharge d'opérateurs, séparation des responsabilités entre classes et tests unitaires indépendants.

Les TP5 et 6 complètent le simulateur en trois apports : des conditions aux limites (réflexion, périodicité, absorption) et le rescaling pour la thermalisation ; 32 tests Google Test garantissant la robustesse physique et numérique ; et quatre diagrammes UML documentant l'architecture statique, les acteurs, la dynamique temporelle et le cycle de vie de la simulation.